

RUPTURE — Préparation Google Play

Rapport technique — packaging Android réel, monétisation sandbox testable, et verdict de préparation à la publication

Dépôt : `scuillerjack-oss/rupture-v0` · Branche : `main`

Commits livrés : `236300c` (Android + monétisation), `214a8e6` (correctif CI Node)

Bêta web permanente (inchangée) : <https://scuillerjack-oss.github.io/rupture-v0/>

1. État initial trouvé (audit préalable)

Avant toute modification, le dépôt était à jour au commit `0481741` (rapport V5.2 du cycle précédent, déjà déployé avec succès). L'audit a confirmé que la stratégie d'empaquetage Android était déjà actée dans `docs/TRAJECTOIRE_COMMERCIALE.md` : **Capacitor**, choisi explicitement plutôt que TWA (Trusted Web Activity) précisément parce que Premium (Play Billing) et publicité (AdMob) nécessitent de vrais plugins natifs, que TWA ne peut offrir qu'à travers des API web encore inégales.

État concret trouvé : dépendances Capacitor déjà présentes dans `package.json` (`@capacitor/core`, `@capacitor/android`, `@capacitor/cli`), `capacitor.config.json` déjà configuré (`appId: com.scuillerjackoss.rupture`), mais `npx cap add android` jamais exécuté — le projet natif n'existait pas encore. Le service Premium/Ads (`src/services/premium.js`, `ads.js`) suivait déjà une architecture « réel / simulé / préparé » propre et testée, avec un point d'indirection unique pour brancher un jour un vrai SDK. Rien de tout cela n'a été reconstruit : cette passe part strictement de cet existant.

Décision Android confirmée, pas re-choisie : l'audit n'a trouvé aucune raison de remettre en cause Capacitor (tâche dédiée de cette passe) — le raisonnement de `TRAJECTOIRE_COMMERCIALE.md` reste valide et à jour au regard des exigences Play actuelles (voir §3).

2. Projet Android réel — ce qui a été préparé

Élément	Détail
Génération	<code>npx cap add android</code> exécuté (opération locale, ne nécessite pas le SDK Android). Décision changée par rapport aux phases précédentes : <code>android/</code> est désormais committé (plus dans <code>.gitignore</code>), car cette passe y ajoute de vraies configurations à préserver (versioning, signature, plugin natif local) plutôt que de le régénérer à l'identique à chaque fois.
applicationId	<code>com.scuillerjackoss.rupture</code> — conservé tel quel (déjà cohérent partout). Reste un PLACEHOLDER à confirmer avant la première soumission (permanent une fois publié).
Versioning	<code>versionCode 1, versionName "5.2.0"</code> (<code>android/app/build.gradle</code>) — à incrémenter manuellement à chaque soumission future (Play Console l'exige).

compileSdk / targetSdk	36 (générés par le template Capacitor 8 actuel) — au-delà de l'exigence Play en vigueur (cible l'API la plus récente), aucun ajustement nécessaire.
Signature	Lit <code>android/keystore.properties</code> (jamais committé, .exemple fourni avec la commande <code>keytool</code> exacte). Absent ici : le build « release » retombe sciemment sur la signature de debug — compile et s'installe, mais n'est PAS soumissible à Google Play tel quel.
Icônes / écran de démarrage	Générés réellement (<code>@capacitor/assets</code>) depuis l'icône PWA maskable existante, sur le fond sombre de la charte (<code>#0b0f14</code>) — pas un simple placeholder générique Capacitor.
Orientation	Verrouillée portrait au niveau natif, cohérent avec <code>manifest.webmanifest</code> .
Permissions	Seule INTERNET (nécessaire pour AdMob/Play Billing) — aucune permission sensible.
Cycle de vie / sauvegarde	Aucun changement nécessaire (confirmé, pas supposé) : la WebView Capacitor utilise le même <code>localStorage</code> et le même <code>document.hidden</code> déjà gérés par <code>src/main.js</code> — le registre de migrations existant (<code>src/engine/migrations.js</code>) couvre déjà le cas d'une mise à jour d'app qui ne doit pas perdre la progression.

2.1. Build réellement vérifié — pas supposé

Limite honnête de cet environnement de développement : sa politique réseau bloque `dl.google.com` (dépôt Maven officiel d'Android). Une première tentative de `./gradlew assembleDebug` ici a échoué dès la résolution du plugin Android Gradle lui-même : `Could not GET 'https://dl.google.com/dl/android/maven2/.../gradle-8.13.0.pom'. Received status code 403 from server: Forbidden`. Ce n'est pas une limite du projet — c'est une limite réseau de cette machine de développement précise.

Solution retenue : un workflow GitHub Actions dédié (`.github/workflows/android-build.yml`), qui compile le projet sur les runners GitHub — une machine différente, avec le SDK Android préinstallé et un accès réseau normal. C'est réellement « le cloud » demandé, indépendant du téléphone/de la machine de l'utilisateur.

Résultat réel et vérifié (pas une simulation de succès) : après un premier échec en CI diagnostiqué et corrigé (le CLI Capacitor 8 exige Node ≥ 22 , le workflow utilisait Node 20 comme `deploy.yml` — qui n'appelle jamais ce CLI et n'a donc jamais ce problème), le workflow [run 34843120864](#) a réussi intégralement : `npm test` (96/96), `npm run build`, `npx cap sync android`, puis trois builds Gradle réels (`assembleDebug`, `bundleDebug`, `bundleRelease`). Deux artefacts téléchargeables ont été produits et vérifiés présents : un paquet debug (APK + AAB, 16,8 Mo) et un AAB release signé en debug (7,1 Mo, « *unsigned-for-play* » dans son nom — pas soumissible tel quel, voir §2).

3. Monétisation réellement testable en sandbox

Point de départ explicite de l'audit : `src/services/premium.js/ads.js` suivaient déjà, avant cette passe, une distinction propre « réel / simulé / préparé ». Rien n'a été reconstruit — les deux fonctions déjà en place (`purchasePremium`, `restorePurchases`, `showInterstitialAd`) ont été étendues pour appeler un vrai SDK quand les conditions sont réunies, en conservant leur comportement existant (simulé en dev/bêta, not -connected par défaut en commercial) dans tous les autres cas.

3.1. AdMob — réellement branché, sandbox utilisable immédiatement

`src/services/adapters/admob.js` (plugin `@capacitor-community/admob`) : appelé réellement sur build Android native + environnement commercial. Utilise par défaut l'unité de test interstitielle officielle de Google (`ca-app-pub-3940256099942544/1033173712`) et l'App ID de test officiel (`ca-app-pub-3940256099942544~3347511713`) — documentés publiquement par Google, jamais un identifiant inventé. Conséquence concrète : **la publicité sandbox fonctionne dès l'installation de l'app, sans aucun compte AdMob** — un vrai SDK, une vraie requête réseau vers les serveurs de test de Google, jamais facturable. Passer en publicité réelle ne demande que de renseigner `VITE_ADMOB_INTERSTITIAL_UNIT_ID/ADMOB_APP_ID` une fois le compte créé.

3.2. Play Billing — plugin natif local, sandbox via license testers

Un composant tiers existant a été audité et rejeté avec une raison précise, pas devinée. Le seul plugin Capacitor Play Billing librement disponible et à jour trouvé (`capacitor-billing`, un seul mainteneur) a été inspecté au niveau de son code natif Java, pas seulement de sa documentation : il s'est révélé incomplet pour ce cahier des charges — aucune méthode de restauration des achats (`queryPurchasesAsync` jamais appelé), et une méthode `finishTransaction` déclarée côté JS mais jamais implémentée côté natif (aurait échoué silencieusement à l'exécution).

Plutôt que de brancher un composant connu incomplet, un petit plugin Capacitor **local** a été écrit directement sur Google Play Billing Library 7.1.1 :

`android/app/src/main/java/.../RuptureBillingPlugin.java` (détails produit, achat, restauration — `queryActivePurchases`, acquittement), exposé côté JS par `src/services/adapters/playBilling.js`. Compile avec succès en CI (voir §2.1).

Ce que le sandbox Play Billing exige, et que rien ne peut contourner : contrairement à AdMob, Play Billing n'a pas d'identifiant de test universel — il exige TOUJOURS qu'un produit existe dans la Play Console pour l'applicationId/la signature exacts de l'app installée. Le mécanisme de test officiel de Google est la liste des **license testers** (Play Console > Configuration > Test de licence) : un compte Gmail qui y figure peut acheter le produit réel sans jamais être facturé, dans des

conditions par ailleurs identiques à un achat réel (même flux, même vérification, même attribution). C'est un mécanisme sandbox *officiel de Google*, pas une simulation construite pour cette passe — et il ne peut être activé que depuis votre compte Play Console (voir §5).

3.3. Publicité récompensée — ré-auditée, confirmée hors scope

La demande mentionnait explicitement de tester « les publicités récompensées et l'attribution exacte de leur récompense lorsqu'elles sont prévues ». Vérification faite sur le modèle économique réellement retenu (docs/RUPTURE_Audit_Economique_Precommercialisation.pdf) : **aucune mécanique de publicité récompensée n'y est définie** — seul l'interstitiel est retenu. Rien n'a donc été branché pour ce point, afin de ne pas fabriquer une mécanique de jeu qui n'a jamais été décidée. L'interface reste prête (isRewardedAdAvailable/showRewardedAd, toujours false/ not-implemented) si une décision produit future en crée une — le plugin AdMob déjà branché expose aussi les méthodes reward-interstitial nécessaires.

3.4. Ce qui reste garanti ne jamais se produire accidentellement

- Web/PWA (bêta permanente) : aucun changement de comportement — toujours env==='beta', jamais natif, donc toujours entièrement simulé comme avant cette passe (vérifié par la suite E2E complète rejouée sans régression, voir §4).
- Android sans VITE_PLAY_PRODUCT_ID/unité AdMob réelle configurée : retombe sur les identifiants de test (AdMob) ou refuse explicitement d'agir (not-connected, Billing) — jamais une simulation qui prétendrait avoir réussi un achat.
- Le code des deux adaptateurs natifs est chargé en *import dynamique* : absent du bundle web/PWA (vérifié par la taille du bundle, inchangée pour le chemin principal, avant/après).

4. Non-régressions vérifiées

Vérification	Résultat
Tests unitaires (node:test)	96/96 verts (92 existants + 4 nouveaux : import et sélection d'unité AdMob, refus Billing sans productId)
Build web de production	propre, rejoué après chaque changement
PWA (manifest, service worker)	inchangée, revérifiée
Mobile (390px)	aucun débordement horizontal
Sauvegarde/reprise	partie jouée → rechargement complet → reprise fonctionnelle, revérifié sur le nouveau bundle

Réglages (Musique/Effets)	fonctionnels, revérifiés
Parcours de monétisation (test, web)	achat/restauration/désactivation Premium simulés, fonctionnels sans erreur
Thème musical (V5.2)	planification revérifiée sur le nouveau bundle - alternance et silences intacts
Erreurs console/page durant tout le parcours	0
Taille du bundle web principal	39,6 Ko → 49,7 Ko (+10 Ko, +25%) - coût du chargement statique de <code>@capacitor/core</code> (nécessaire pour <code>isNativePlatform()</code>) ; AdMob/Billing eux-mêmes sont en chunks séparés chargés uniquement sur Android natif (3,3 Ko et 1,1 Ko, jamais téléchargés par un visiteur web)

5. Déploiement

- Commits poussés sur `origin/main` : 236300c (Android + monétisation), 214a8e6 (correctif Node du workflow CI Android).
- **Bêta web permanente** : workflow `Deploy to GitHub Pages` vert sur les deux commits (runs [34842940440](#) et [34843120897](#), conclusion *success*) — le canal de bêta actuel n'a subi aucune interruption ni régression pendant toute cette transition.
- **Build Android** : workflow `Build Android app` vert sur le second commit (run [34843120864](#), conclusion *success*), artefacts APK/AAB réellement produits et vérifiés présents (voir §2.1). Ce workflow ne publie rien sur aucun store — il construit et conserve les artefacts 90 jours pour inspection/test, aucune action de publication automatisée n'existe.

6. Verdict de préparation Google Play

6.1. Déjà prêt (validé avant cette passe, inchangé)

- **PRÊT** Gameplay stabilisé par plusieurs cycles de bêta (V0 → V5.2), moteur testé (96 tests unitaires + batteries de simulation).
- **PRÊT** Aucune permission Android sensible requise.
- **PRÊT** Aucune donnée envoyée à un serveur tiers en dehors d'AdMob/Play Billing eux-mêmes une fois activés (aucun backend, aucun compte joueur).

6.2. Préparé par cette passe (nouveau, vérifié)

- **PRÉPARÉ** Projet Android réel généré, configuré, compile et s'installe (APK/AAB produits en CI).
- **PRÉPARÉ** AdMob réellement branché, sandbox fonctionnelle sans compte.

- **PRÉPARÉ** Play Billing réellement branché (plugin local complet), sandbox utilisable dès qu'un produit + des license testers existent côté Play Console.
- **PRÉPARÉ** Icônes/splash réels, versioning, structure de signature.

6.3. Manque obligatoirement avant toute publication réelle

- **MANQUANT** Un keystore d'upload réel et signé (le build actuel retombe sur une signature de debug, refusée par Play Console).
- **MANQUANT** Politique de confidentialité hébergée publiquement (obligatoire dès qu'AdMob/Billing sont actifs).
- **MANQUANT** Formulaire « Sécurité des données » (Data safety) à jour — doit désormais déclarer AdMob et Play Billing comme SDK connectés dès qu'un vrai compte existe.
- **MANQUANT** Questionnaire de classification par âge/contenu, fiche store (description, captures d'écran).
- **MANQUANT** Consentement publicitaire RGPD/EEE (SDK "User Messaging Platform" de Google) si l'audience européenne est ciblée.

6.4. Nécessite une action de votre part (comptes, décisions, ne peut être fait depuis cet environnement)

Action	Pourquoi elle ne peut pas être faite ici
Compte développeur Google Play (~25 \$US, unique)	Compte personnel/société, paiement réel
Génération du keystore d'upload (<code>android/keystore.properties.example</code> documente la commande exacte)	Doit être créé et conservé par vous - sa perte est irréversible pour les mises à jour futures
Compte AdMob + vraie unité publicitaire + <code>ADMOB_APP_ID</code>	Compte personnel, nécessaire uniquement pour la publicité réelle (le sandbox fonctionne déjà sans)
Produit "Premium" dans la Play Console + <code>VITE_PLAY_PRODUCT_ID</code>	Play Billing refuse tout produit qui n'existe pas côté Play Console pour votre app signée - condition du protocole, pas de ce code
License testers (Play Console)	Liste d'adresses Gmail vous appartenant - mécanisme sandbox officiel de Google
Confirmation finale de l' <code>applicationId</code>	Décision produit permanente une fois publiée
Politique de confidentialité, Data safety, classification, fiche store	Décisions de contenu/juridiques, pas des sujets techniques

7. Points à surveiller lors de la prochaine bêta/étape

1. **Taille du bundle web** (+10 Ko sur le chemin principal) : acceptable, mais à surveiller si d'autres dépendances natives sont ajoutées plus tard - envisager de charger `Capacitor.isNativePlatform()` via une détection plus légère si cela devient significatif.
2. **Icône adaptative Android** générée automatiquement depuis l'icône PWA plate - fonctionne, mais une retouche graphique premier-plan/arrière-plan séparée donnerait un rendu plus abouti (décision visuelle non prise ici, voir §2).
3. **Play Billing en conditions réelles** : ce plugin local n'a pu être vérifié qu'à la compilation (CI) - un premier achat réel via un license tester, une fois la Play Console configurée, reste le test le plus fiable pour confirmer le flux complet (achat → acquittement → persistance → restauration après réinstallation).
4. **Build release non signé** : ne pas confondre l'artefact `rupture-android-release-unsigned-for-play` produit par la CI (utile pour vérifier que le projet compile) avec un artefact réellement soumissible - il ne l'est pas tant qu'un vrai keystore n'existe pas.

RUPTURE reste volontairement plus simple que FRONTIÈRES : aucune sur-architecture n'a été introduite dans cette phase - le moteur de jeu et l'interface n'ont subi aucune modification.